# C05 - Assessment Tasks 2


## Name: N.R.Yasaswini

## Register Number: 192425119


## Error Analysis Task

## SET1

### 1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

The likely design error is poor input partitioning or an inappropriate split configuration. If input files are highly uneven in size, Hadoop creates partitions that result in some mapper tasks receiving much more data than others. This causes data skew and creates a straggler effect: most tasks finish early while a few large tasks continue running. Reducers may also remain idle if the mapper output is poorly balanced or if the job is configured with an unsuitable number of reducers.

**Corrective actions:**

- Inspect input-file sizes and avoid a large number of tiny files as well as extremely large unbalanced files.
- Repartition or rebalance the input data so that splits are reasonably uniform.
- Configure an appropriate HDFS block size and Hadoop input format/split settings.
- Use CombineFileInputFormat when many small files create inefficient mapper allocation.
- Examine partitioning keys for skew and use a custom partitioner or salting technique when one key receives disproportionately large data.
- Choose the number of reducers according to data volume, cluster capacity, and expected reducer workload.
- Enable speculative execution where appropriate to reduce the effect of slow or straggling tasks.
- Monitor mapper and reducer task distribution through YARN/JobHistory tools.

Thus, the core problem is data or workload skew caused by poor partitioning. Balanced input splits and appropriate reducer partitioning allow tasks to execute more evenly, reducing job completion time and preventing idle cluster resources.

## 2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

HDFS is designed to tolerate DataNode failures through block replication. Frequent data unavailability after only one DataNode fails therefore indicates a serious replication or metadata availability problem.

**Probable replication errors include:**

- The replication factor may have been configured too low, such as one replica for important data.
- Replicas may have been placed on insufficient or poorly distributed DataNodes.
- Rack awareness may not be configured correctly, causing replicas to be concentrated in the same failure domain.
- Replication may be incomplete because failed or decommissioned nodes were not replaced and blocks were left under-replicated.
- HDFS health monitoring may not be detecting under-replicated blocks quickly enough.

A NameNode design error may also be involved. If the deployment uses only a single NameNode without a properly configured standby NameNode, NameNode failure can make the filesystem namespace unavailable even though DataNode blocks still exist. In a highly available design, an Active and Standby NameNode should share metadata through JournalNodes or an equivalent shared-edit-log mechanism, with ZooKeeper-based coordination/failover where applicable.

**Corrective actions:**

- Use an appropriate replication factor, commonly three for important production datasets.
- Enable rack-aware replica placement.
- Monitor and repair under-replicated blocks.
- Use NameNode High Availability with Active/Standby NameNodes.
- Maintain reliable metadata backups/checkpoints and test recovery procedures.
- Monitor DataNode health, disk failures, capacity, and decommissioning status.

Therefore, the most likely design flaw is insufficient or poorly distributed replication, possibly combined with a single point of failure at the NameNode layer.

## 3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Duplicate records usually occur when the ingestion workflow is not idempotent or when NiFi processes the same data more than once. ETL retries, overlapping schedules, replayed messages, and weak record-identification logic are common causes.

**Likely causes:**

- NiFi processors retry a failed flow after the destination has already accepted the record.
- A source file or message is picked up repeatedly because it is not marked, moved, or tracked correctly.
- Multiple NiFi flows ingest the same source data.
- Network failures cause an acknowledgement to be lost, leading to retransmission.
- The analytics store performs blind INSERT operations without checking whether the record already exists.
- No unique business key or event ID is used to identify individual records.
- Parallel processing creates race conditions when two flow paths process the same record.

**Recommended fixes:**

- Give every event a stable unique identifier and use it as an idempotency key.
- Configure NiFi to manage file completion, provenance, retries, and failure queues carefully.
- Use appropriate processors and flow design to prevent the same source object from being consumed repeatedly.
- Implement upsert/merge logic rather than unconditional INSERT operations.
- Add database primary keys or unique constraints based on the correct business key.
- Maintain ingestion checkpoints or offsets for streaming sources.
- Separate successful, failed, and retry paths so that a failed transaction does not create a second successful copy.
- Use deduplication logic where necessary before loading the analytics store.
- Monitor NiFi provenance and destination audit logs to identify the exact point where duplication occurs.

The key principle is idempotent ingestion: processing the same input more than once should not create additional logical records.

## 4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

The likely scheduling error is poor resource allocation or an inappropriate YARN scheduler configuration. If all users submit jobs into a single queue without capacity limits, one or a few applications can consume most CPU and memory resources, causing starvation for other users.

**Problems may include:**

- No queue hierarchy or resource quotas.
- Incorrect Capacity Scheduler or Fair Scheduler configuration.
- Excessive resource requests by individual applications.
- Missing user or queue limits.
- Poorly configured container memory and CPU settings.
- Lack of application priorities and preemption policies.
- Overcommitment of cluster resources.

A better approach is to use YARN's queue-based resource management. For example, separate queues can be created for departments, teams, or workload types. Each queue can receive a defined capacity, maximum capacity, and user/application limits.

**Corrective actions:**

- Configure the Capacity Scheduler or an appropriate scheduler according to organizational needs.
- Define minimum and maximum resources for queues.
- Apply per-user and per-application limits.
- Configure queue priorities where urgent workloads require preferential access.
- Enable controlled preemption so that a queue consuming excess resources can release resources when another queue needs them.
- Set appropriate container CPU and memory limits.
- Monitor queue utilization and application resource consumption.
- Tune scheduler settings based on actual workload patterns.

For example, a cluster can reserve capacity for production analytics while allowing development jobs to use remaining resources. This prevents one user or workload from monopolizing the cluster.

Therefore, the solution is not simply adding hardware; it is implementing fair, controlled, and policy-based resource scheduling so that multiple users receive predictable access to cluster resources.

## 5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

The probable design flaw is that the backup system is restoring from an old or stale copy instead of the most recent valid recovery point. This can happen when replication is asynchronous, backup jobs run too infrequently, snapshots are not verified, or the recovery process selects the wrong backup version.

**Possible design flaws:**

- Backups are scheduled too infrequently, creating a large Recovery Point Objective (RPO).
- Replication lag is not monitored.
- A secondary replica is treated as a current backup even though it is stale.
- Snapshots are overwritten without maintaining historical recovery points.
- Backup metadata or timestamps are unreliable.
- The recovery procedure does not select the latest successful backup.
- Backups are stored in the same failure domain as the primary data.
- Backup integrity is not regularly tested.

**Corrective actions:**

- Define explicit RPO and Recovery Time Objective (RTO) requirements.
- Use frequent incremental backups and periodic full backups according to business needs.
- Monitor replication lag and backup-job success.
- Maintain multiple recovery points rather than only the latest snapshot.
- Use versioning and immutable backups to protect recovery copies from accidental deletion or corruption.
- Store backups in a separate availability zone or geographic region.
- Verify backup integrity through automated validation.
- Perform regular restore tests to confirm that backups are actually recoverable.
- During recovery, select the latest verified backup that satisfies the required consistency point.

For critical distributed storage, replication and backup should not be treated as identical mechanisms. Replication provides availability and rapid recovery, while independent versioned backups protect against corruption, accidental deletion, and recovery from an earlier point in time.

Hence, the central flaw is inadequate recovery-point management—using stale or unverified replicas/backups without monitoring replication freshness and testing restoration.